Øving 10: Korteste vei fra én til alle

Kollapser kontekst

Du må oppnå minst 10 av totalt 23 mulige poeng for å denne øvingen godkjent. Hver flervalgsoppgave er verdt 1 poeng, mens hver kodeoppgave er verdt 3 poeng. For informasjon om øvingsopplegget, se **nettsidene**. Trenger du hjelp, så **ta kontakt**.

Løsningsforslaget publiseres på **Canvas** etter innleveringsfristen.

Det er noen feil i din innlevering. Du fikk 34.78%. [Submission #6ae8aedfdc3bc30277398630 - **02.11.2026, 16:40:15**]

Spørsmål 1: Refleksjon

Dette er en refleksjonsoppgave. Denne oppgaven teller ikke mot antall oppgaver som må løses for å få godkjent øvingen. For mer informasjon om refleksjonsoppgavene, se **nettsidene**.

Refleksjonsoppgavene handler kun om refleksjon rundt forståelse av og arbeid med fagstoffet og øvingen. Pass på så du ikke inkluderer noen **sensitive personopplysninger** i refleksjonen din.

Før du begynner å gjøre øvingen, reflekter over følgende:

- Hvilke deler av pensum knyttet til denne øvingen er du usikker på?
- Hva tror du at du kommer til å lære ved å gjøre denne øvingen?

Etter du er ferdig med øvingen, reflekter over følgende:

- Hva forstår du nå, som du ikke forsto før du begynte å jobbe med øvingen?
- Hvilke nye sammenhenger ser du?
- Kan du komme på flere anvendelser av stoffet fra denne øvingen? I så fall hvilke?
- Er det noe du fortsatt er usikker på? I så fall hva?
- Hvis du skulle gjort øvingen på nytt, hva ville du gjort annerledes?

Spørsmål 2: Korteste vei fra én til alle

✓ Riktig svar

Hva går korteste-vei-problemet (fra én til alle) ut på?

- ○ Finne sykler med færrest mulig kanter.
- ◉ Finne stier som minimerer summen av kantvektene.
- ○ Finne sykler som minimerer summen av kantvektene.
- ○ Finne stier med færrest mulig kanter.

Spørsmål 3: Korteste vei fra én til alle

✓ Riktig svar

Hvilke påstander er korrekte?

- [x] Selv om man har en kortest vei fra node A til B og en kortest vei fra node B til C, vil man ikke kunne garantere at det finnes en kortest vei mellom A og C som går via B.
- [ ] Et dybde-først-søk vil finne korteste vei til alle noder i en graf.
- [ ] Korteste-vei-problemet har ikke optimal delstruktur hvis vi har positive kantvekter.
- [ ] Det finnes aldri mer enn én kortest vei mellom to noder.
- [x] Ved å snu alle kantene i en graf, kan man finne korteste veier fra alle til én, gitt at man vet hvordan man finner korteste veier fra én til alle.

Spørsmål 4: Korteste vei fra én til alle

✓ Riktig svar

Anta at du har funnet korteste vei mellom nodene A og C, og at den går gjennom node B. Hvilken påstand er korrekt?

- ○ Vi har også funnet korteste vei mellom A og B, men ikke mellom B og C.
- ◉ Vi har også funnet korteste vei mellom A og B og mellom B og C.
- ○ Vi har også funnet korteste vei mellom B og C, men ikke mellom A og B.
- ○ Ingen av de andre påstandene er korrekte.

Spørsmål 5: Korteste vei fra én til én

✓ Riktig svar

Hvorfor ser vi ikke på korteste vei fra én til én i pensum?

- ◉ Man har ikke algoritmer som i verste tilfelle løser dette problemet raskere enn korteste vei fra én til alle.
- ○ Det er aldri raskere å løse dette enn å løse fra én til alle.
- ○ Det er aldri et relevant problem å løse.

Spørsmål 6: Korteste vei fra én til alle

✓ Riktig svar

Hvilke påstander er korrekte?

- [ ] Bellman-Ford finner alltid korteste vei med både negative kanter og negative sykler.
- [x] Bellman-Ford finner alltid korteste vei med negative kanter, men ikke negative sykler.
- [ ] Dijkstra finner alltid korteste vei med både negative kanter og negative sykler.
- [x] Vi kan fjerne 0-vektede sykler fra en sti for å lage en ny sti med samme vekt.
- [ ] Dijkstra finner alltid korteste vei med negative kanter, men ikke negative sykler.
- [ ] Det er umulig å detektere om en graf har negative sykler.

Spørsmål 7: Relax

✓ Riktig svar ×

Vi har $u.d = 3$, $v.d = 7$, $w(u, v) = 3$. Hva blir $u.d$ og $v.d$ etter et kall til $\textsc{Relax}(u, v, w)$?

- ○ $u.d = 6, v.d = 6$
- ○ $u.d = 10, v.d = 6$
- ○ $u.d = 3, v.d = 7$
- ○ $u.d = 6, v.d = 10$
- ◉ $u.d = 3, v.d = 6$
- ○ $u.d = 6, v.d = 10$
- ○ $u.d = 6, v.d = 7$
- ○ $u.d = 3, v.d = 10$

Spørsmål 8: Relax

✓ Riktig svar ×

Vi har $u.d = 3$, $v.d = 7$, $w(u, v) = 3$. Hva blir $u.\pi$ og $v.\pi$ etter et kall til $\textsc{Relax}(u, v, w)$?

- ○ $u.\pi = v, v.\pi = \text{NIL}$
- ○ $u.\pi = \text{NIL}, v.\pi = \text{NIL}$
- ◉ Vi kan ikke vite hva $u.\pi$ er gitt informasjonen i oppgaven, men $v.\pi = u$.
- ○ $u.\pi = u, v.\pi = v$
- ○ Vi kan ikke vite hva $v.\pi$ er gitt informasjonen i oppgaven, men $u.\pi = v$.
- ○ Vi kan hverken vite hva $u.\pi$ eller $v.\pi$ er gitt informasjonen i oppgaven.
- ○ $u.\pi = \text{NIL}, v.\pi = u$
- ○ $u.\pi = v, v.\pi = u$

Spørsmål 9: Korteste vei fra én til alle

✓ Riktig svar ×

Anta at du har kjørt en av algoritmene i pensum som løser korteste-vei-problemet, fra én til alle, for grafen $\mathrm{G} = (\mathrm{V}, \mathrm{E})$, med startnode s og vekter $w(u, v) \in \mathbb{R}$. Hvilke av disse er mulige verdier for $a.d$ for en $a \in \mathrm{V}$?

- ☐ NIL
- ☑ 0
- ☑ $\sum_{(u,v)\in \mathrm{P}} w(u, v)$, hvor P er den korteste stien mellom s og a
- ☐ $-\infty$
- ☑ ∞
- ☑ -1

Spørsmål 10: Byggetid

✗ Koden krasjet.

```
  File "building_time.py", line 2
    # Skriv din kode her
                        ^
IndentationError: expected an indented block after function definition on line 1
```

Du eier et byggefirma som spesialiserer seg på å bygge så raskt som mulig. Dette er noe du løser ved å la flere deler av byggingen skje i parallell, men uheldigvis er det begrenset hvor mye det hjelper å ha flere bygningsarbeidere, siden noen oppgaver må gjøres før andre. For eksempel kan man ikke legge inn elektrisitet før veggene er bygd, og veggene kan ikke bygges før fundamentet til bygget er lagt. Du har flere byggeprosjekter, og for å forenkle utregningene ønsker du å lage en funksjon som forteller deg hvor lang tid det tar å bygge et bygg dersom du har ubegrenset med arbeidskraft.

Du skal implementere funksjonen `building_time(tasks)`, der `tasks` er en liste med `Task`-objekter som representerer de ulike oppgavene som må utføres under byggingen. Et `Task`-objekt har følgende attributter:

- `time` – Antall dager det tar å utføre oppgaven.
- `depends_on` – En liste med `Task`-objekter som må utføres før denne oppgaven kan påbegynnes.

Funksjonen skal returnere hvor lang tid det tar å bygge bygget, i antall dager, dersom man starter med en oppgave i det alle oppgavene den er avhengig av er ferdigstilte. Du kan anta at det alltid er mulig å bygge bygningen. Kjøretiden til funksjonen bør være $\Theta(V + E)$.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

```
1  def building_time(tasks):
2      # Skriv din kode her
```

Spørsmål 11: Dag-Shortest-Paths kontra Dijkstra

✗ Galt svar

DAG-SHORTEST-PATHS krever at grafen den kjøres på er rettet og asyklisk. DIJKSTRA krever ikke dette. Hva er likevel fordeler med å bruke DAG-SHORTEST-PATHS? (Anta at vi bruker DIJKSTRA med Fibonacci-haug.)

- ☐ DAG-SHORTEST-PATHS har bedre asymptotisk tidskompleksitet enn DIJKSTRA i verste tilfelle uansett hva $|E|$ er.
- ☐ DAG-SHORTEST-PATHS har bedre asymptotisk tidskompleksitet enn DIJKSTRA i verste tilfelle om $|E| = o(V \lg V)$.
- ☐ DAG-SHORTEST-PATHS fungerer også med negative kantvekter, mens DIJKSTRA gjør ikke det.
- ☐ DIJKSTRA fungerer ikke på rettede asykliske grafer, selv om de har positive kantvekter.

Spørsmål 12: Dijkstra - prioritetskø

? Ingen svar

Hvilken operasjon har amortisert kjøretid $\Theta(\lg n)$ i verste tilfelle i en prioritetskø implementert med en binærhaug, men har amortisert kjøretid $O(1)$ i verste tilfelle i en prioritetskø implementert med en Fibonacci-haug?

- ○ EXTRACT-MIN
- ○ MINIMUM

○ Decrease-Key

Spørsmål 13: Minst energi

✕ Koden krasjet. ✕

```
  File "least_energy.py", line 7
    highscore = False
    ^^^^^^^^^
IndentationError: expected an indented block after function definition on line 1
```

Du er kjemiker på et laboratorium der dere prøver å finne den beste måten å fremstille et bestemt stoff fra et annet stoff. Dette gjøres ved å frembringe kjemiske reaksjoner som gjør et stoff om til et annet stoff. De kjemiske reaksjonene er enten eksoterme eller endoterme, som betyr at de henholdsvis slipper ut energi til eller henter inn energi fra omgivelsene. Laben må tilføre energi (typisk varme) for endoterme reaksjoner, men kan også hente ut energi fra eksoterme reaksjoner. For å gjøre prosessen mest mulig miljømessig og økonomisk gunstig, ønsker dere å finne den måten å fremstille stoffet på som krever lavest netto energi tilført fra laben.

Du skal implementere funksjonen `least_energy(reactions, start, goal, laws_of_thermodynamics=True)`, hvor `reactions` er en liste med tupler på formen `(a, b, e)`. Et slikt tuppel betyr at man kan gå fra stoff `a` til stoff `b` (men ikke motsatt) ved hjelp av en reaksjon som må tilføres netto `e` enheter energi. Her er `a` og `b` tekststrenger, og `e` er et heltall som er positivt for endoterme reaksjoner og negativt for eksoterme reaksjoner. Hvert ordnede par `(a, b)` forekommer kun én gang. Parametrene `start` og `goal` er tekststrenger som representerer stoffet du henholdsvis starter med og skal ende opp med. Funksjonen skal returnere en liste `s` der `s[0] = start`, `s[-1] = goal`, og der vi går fra stoff `s[i]` til stoff `s[i+1]` for hvert trinn `i` på veien. Parameteren `laws_of_thermodynamics` brukes kun til poengtavlen og kan ignoreres her.

Du er garantert at det finnes en måte å fremstille `goal` fra `start`. Termodynamikkens lover garanterer deg at det ikke finnes noen måte å gå fra et stoff til det samme stoffet via andre stoffer slik at summen av netto tilført energi fra laben er negativ.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Nå er `laws_of_thermodynamics = False`, noe som betyr at at det er mulg å gå fra et stoff til det samme stoffet via andre stoffer slik at summen av netto tilført energi fra laben er negativ. Dette betyr at du potensielt kan generere så mye energi du vil! Siden du ikke er i kraftbransjen, men heller prøver å fremstille et stoff, vil du sikre at du har fremdrift hele tiden. Listen du returnerer kan derfor inneholde hvert stoff makismalt én gang.

Du får her poeng basert på summen av netto energi tilført laben for hver reaksjon funksjonen din returnerer.

```python
def least_energy(reactions, start, goal, laws_of_thermodynamics=True):
    # Skriv din kode her

# Sett 'highscore' til True hvis du vil vises på poengtavlen.
# For mer info se 'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
# Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
highscore = False
```

Spørsmål 14: Korteste togrute

✕ Koden krasjet. ✕

```
  File "earliest_arrival.py", line 2
    # Skriv din kode her
                        ^
IndentationError: expected an indented block after function definition on line 1
```

Endelig er ekspresstognettet i Europa godt nok utbygget til at det er mulig å bruke det for å reise rundt. Et ekspresstog går direkte mellom to byer uten noen stopp på veien, noe som sparer en mye tid. Det som fortsatt mangler er en reiseplanlegger som gir deg raskeste rute mellom to byer, noe du skal implementere i denne oppgaven.

Du skal implementere en funksjon `earliest_arrival(timetable, start, goal)` hvor `timetable` er en liste på formen `(a, b, t_0, t_1)` som representerer at det går et tog fra byen `a` til byen `b` med avreise ved tidspunkt `t_0` og ankomst ved tidspunkt `t_1`. Her er `a` og `b` tekststrenger, mens `t_0` og `t_1` er heltall med `t_0 < t_1`.

Passasjeren starter i byen `start` ved tidspunkt `0`. Funksjonen skal returnere tidspunktet når passasjeren tidligst kan ankomme byen `goal` ved å ta en serie med ekspresstog. Du kan anta at hvis du ankommer en by ved tidspunkt `t` kan du også ta et annet tog som har avreise ved samme tidspunkt. Det vil alltid være mulig å komme seg fra `start` til `goal`.

Hvis det er n byer og m tog, bør funksjonen din ha en kjøretid på $o(nm)$.

Ønsker du å teste programmet ditt lokalt, finnes det eksempeltester her.

```
1  def earliest_arrival(timetable, start, goal):
2      # Skriv din kode her
```

Spørsmål 15: Dijkstra - færrest kanter

Denne oppgaven er basert på en eksamensoppgave.

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Du har en graf G med positive heltallsvekter på kantene. Du vil finne korteste vei fra en node s til alle de andre nodene i grafen, men dersom det står mellom to veier, og den ene har færre kanter enn den andre, skal veien med færrest kanter velges. Hvordan kan du endre kantvektene for at $\textsc{Dijkstra}$ skal finne denne korteste veien?

Spørsmål 16: New-SSSP

? Ingen svar

Du har laget en ny algoritme $\textsc{New-SSSP}$ som er ment å finne korteste vei i en graf G med potensielt negative kantvekter, men uten negative sykler:

```
NEW-SSSP(G, w, s)
 1  INITIALIZE-SINGLE-SOURCE(G, s)
 2  for each vertex v ∈ V
 3       v.queued = FALSE
 4  let Q = ∅ be a FIFO queue
 5  ENQUEUE(Q, s)
 6  s.queued = TRUE
 7  while Q ≠ ∅
 8       u = DEQUEUE(Q)
 9       u.queued = FALSE
10       for each vertex v ∈ G.Adj[u]
11            if v.d > u.d + w(u, v)
12                 v.d = u.d + w(u, v)
13                 v.π = u
14                 if not v.queued
15                      ENQUEUE(Q, v)
16                      v.queued = TRUE
```

Vil denne algoritmen garantert finne korteste vei fra noden s til alle de andre nodene i G?

- ○ Nei
- ○ Ja

Spørsmål 17: New-SSSP

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Kom med et bevis for at NEW-SSSP er korrekt eller eventuelt et moteksempel der algoritmen gir galt svar.

Spørsmål 18: New-SSSP

? Ingen svar ×

Hva er tidskompleksiteten til NEW-SSSP i beste tilfelle?

- ○ $\Theta(\mathrm{V}+\mathrm{E})$
- ○ $\Theta(\mathrm{V})$
- ○ $\Theta(\mathrm{E})$
- ○ $\mathrm{O}(1)$
- ○ $\Theta(\mathrm{VE})$

Spørsmål 19: New-SSSP

? Ingen svar ×

Hva er tidskompleksiteten til NEW-SSSP i verste tilfelle dersom den kjøres på en graf G uten negative sykler?

- ○ $\Theta(\mathrm{E}^2)$

- ○ $\Theta(V^2)$
- ○ $\Theta(VE)$
- ○ $\Theta(V + E)$
- ○ $\Theta(E)$

Spørsmål 20: New-SSSP

? Ingen svar ×

Hva vil resultatet av $\textsc{New-SSSP}$ bli dersom det er negative sykler i grafen?

- ○ Algoritmen vil garantert gi riktig svar.
- ○ Algoritmen vil ikke terminere.
- ○ Algoritmen kan gi feil svar.
- ○ Algoritmen vil ikke terminere dersom en av de negative syklene er mulig å nå fra s, ellers vil den ikke garantert gi riktig svar.
- ○ Algoritmen vil ikke terminere dersom en av de negative syklene er mulig å nå fra s, ellers vil den garentert gi riktig svar.

Spørsmål 21: Hamiltonsti

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Denne oppgaven er et frempek i pensum og kan oppleves som vanskelig. Om du ikke får den til, fortvil ikke.

Du vil finne ut om en graf har en hamiltonsti. En hamiltonsti er en sti som går fra en node til en annen via alle de andre nodene i grafen. Hver node forekommer akkurat én gang i stien. Du har en funksjon $\textsc{Shortest-Simple-Path}(G, w, s, t)$ som finner lengden på korteste enkle sti fra node s til node t i en generell graf med kantvekter $w \in \mathbb{R}$.

Hvordan kan du bruke funksjonen $\textsc{Shortest-Simple-Path}$ til å finne ut om en graf har en hamiltonsti? Klarer du å finne en løsning der du endrer grafen slik at du kun trenger å kjøre $\textsc{Shortest-Simple-Path}$ én gang?

Spørsmål 22: Anbefalte ekstraoppgaver

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Her følger en liste over anbefalte ekstraoppgaver, som kan være nyttig å se på om du ønsker litt ekstra trening eller som repetisjon mot eksamen. Oppgavene er hentet fra gamle eksamensoppgaver og *Introduction to Algorithms* av Cormen mfl. (CLRS), og er sortert basert på kilde.

- 2025H - 11, 16
- 2024H - 17
- 2024S - 7, 13
- 2023S - 4, 16
- 2022H - 1, 10
- 2021H - 9, 10
- 2021S - 2
- 2020S - 4
- 2019H - 1, 6

- 2019S - 13
- 2018H - 13, 14
- 2018S - 8
- 2017H - 1e)
- 2017S - 7
- 2016H - 5
- 2015H - 1, 2, 12
- 2015S - 8
- 2014H - 1a), 2b)
- **CLRS** - 22.1-2, 22.1-3, 22.1-5, 22.1-6, 22.1-7
- **CLRS** - 22.2-2, 22.2-4
- **CLRS** - 22.3-3, 22.3-4, 22.3-6, 22.3-7, 22.3-8, 22.3-9, 22.3-10, 22.3-12

Send inn

Kjører INGInious v.0.9.2.dev240+gb8a04c56.d20260915
© 2014-2026 Université catholique de Louvain.
INGInious distribueres med AGPL-lisens